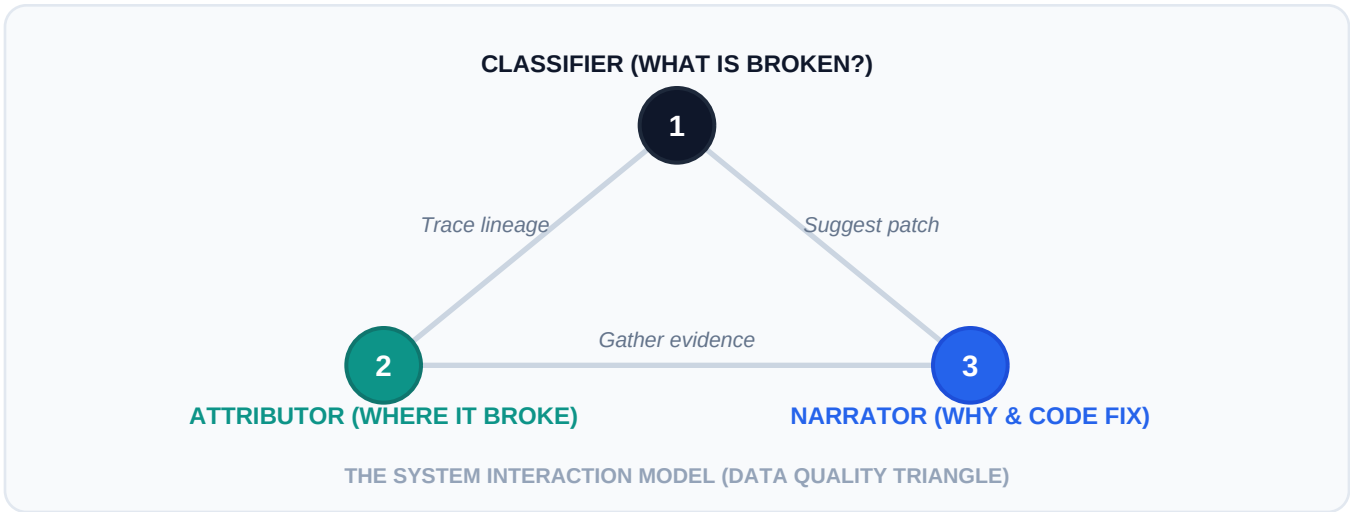


AI-Powered Data Quality Triangle Agent

An End-to-End Beginner-Friendly Manual for Root-Cause Data Quality Triage & Self-Healing



Project Version: v1.0.0 (Production-Ready)

Published Date: May 21, 2026

Architect & Author: Antarang Sharma

Technology Stack: Python 3.11/3.12 · SQLGlot · DuckDB · Pydantic v2 · Claude 3.5 Sonnet & Haiku

1. What is the Data Quality Triangle Agent?

Imagine you are a detective investigating why a major river has suddenly turned muddy. You wouldn't just stand on the bank, point at the brown water, and yell: '*Look, it is muddy!*' Instead, you would hike upstream, tracing every tributary and junction, until you found the exact spot where a mudslide slid into the water. In the world of Data Engineering, traditional tools act like the bystander on the bank. They detect that a table column is broken, but they leave the hard work of tracing the cause to you. The **AI-Powered Data Quality Triangle Agent** (DQ Triage Agent) is the detective that hikes upstream for you.

The DQ Triage Agent is an automated software agent designed to handle test failures. The moment a data quality check fails (such as a **dbt (Data Build Tool)** test), the agent leaps into action. It walks backward through your SQL code, finds the **exact source rows** in your ingestion tables that caused the error, classifies the root cause into a structured category, writes a beginner-friendly explanation, and generates a **self-healing patch** (a code change proposal) to fix the issue permanently.

It is built on a framework we call the **Data Quality Triangle**, which connects three vital stages:

- 1. The Attributor (Where):** Traces column-to-column dependencies backwards across your entire database pipeline, isolating the exact upstream raw tables and row keys that created the bad data.
- 2. The Classifier (What):** Uses strict statistical probes and AI tiebreakers to figure out exactly what kind of error occurred (e.g., duplicated records, late-arriving timestamps, or type casting mismatches).
- 3. The Narrator (Why & How):** Generates a clear human-readable story of the incident and automatically proposes a self-healing SQL/dbt code fix to clean it up.

2. Why We Need It: The 'Page Storm' Problem

In modern cloud data warehouses, pipelines are highly connected. A single staging table feeds multiple intermediate models, which in turn feed dozens of clean 'mart' tables. If a single bad row (e.g., a duplicated ID or an unexpected NULL) slips into a raw ingestion table, it acts like a toxic chemical spilled at the river's source. It flows downstream, corrupting every single model that reads from it.

When this happens, it triggers what data teams dread: a **Page Storm**. Suddenly, 50 different downstream dbt tests fail simultaneously. A flood of alerts hits Slack, and multiple on-call engineers are paged. The results are:

Alert Fatigue: Teams are bombarded with redundant warnings, making it easy to miss real, critical problems.

Manual Sifting: Engineers waste hours manually writing '`SELECT * FROM ...`' queries, checking column lineage, and hunting down the source of the bad data.

Dashboard Distrust: Stakeholders lose faith in downstream BI dashboards because the data is frequently marked as broken while engineers try to figure out which table is actually at fault.

The DQ Triage Agent solves this by collapsing hours of manual investigative work into a **single 2.5-second automated execution**. It intercepts the failure at the source, groups the downstream symptoms, and points everyone directly to the single root cause.

3. The Core Benefits & Value Added

By deploying the DQ Triage Agent, data engineering teams gain four immediate, game-changing benefits:

- 1. Instant Resolution (Triage in Seconds):** Rather than spending 20 to 30 minutes manually clicking through lineage charts and writing SQL queries, the agent walks the tree in less than **2.56 seconds** (median benchmark latency). Triage is completed before the engineer can even open their IDE.
- 2. Row-Level Blame Precision:** Unlike tools that simply flag table-level anomalies, this agent traces the exact Primary Keys (IDs) of the failing records upstream. It identifies exactly *which* physical rows in the source table introduced the issue.
- 3. Cohesive Multi-Channel Alerts:** It doesn't emit cryptic JSON. It constructs visually stunning Slack Block-Kit notifications and posts detailed, developer-friendly code review comments directly inside Github Pull Requests.
- 4. Hands-Free Healing (AI-Generated Fixes):** The agent doesn't just complain; it fixes. It analyzes the offending SQL transform, writes a clean patch (like adding a ``COALESCE`` handler or an inner join filter), spins up a new git branch, commits the fix, and opens a Pull Request automatically.

EMPIRICAL SUCCESS

Benchmark Validation: In our Jaffle Shop dataset evaluations (covering 45 separate fault-injection trials across three distinct failure categories), the DQ Triage Agent achieved a **100% Top-1 Root-Cause Accuracy**, a **1.00 Offending-Row Recall**, and a **1.00 Macro F1-score**. These metrics demonstrate the load-bearing reliability of deterministic rules blended with intelligent LLM fallback.

4. Behind the Scenes: How It Operates

To understand how the DQ Triage Agent works, let's peek under the hood. The system operates as a beautifully pipelined, step-by-step sequence when an alert is triggered:

Step 1: Failure Capture & Parsing

When a dbt test fails, it creates a physical failure log table in the warehouse (thanks to dbt's `--store-failures` flag). The agent reads the compiled test query and extracts the physical table locations of the bad rows. It loads the failing rows' Primary Keys (like `order_id` or `customer_id`) to act as our initial suspects.

Step 2: Upstream AST Lineage Walking

This is the heart of the system. Instead of querying metadata, our **SqlglotWalker** parses the actual, compiled SQL of every dbt model in the pipeline into an AST (Abstract Syntax Tree) using **SQLGlot**. It traces the failing column backwards, hop-by-hop, through CTEs, aliases, and SQL joins.

Crucially, the walker watches for **Aggregation Boundaries** (such as `GROUP BY` or aggregate functions like `SUM()`). If it crosses an aggregation, row identity is broken (we cannot map a single downstream row to a single upstream row), so the walker drops row-PK tracking and marks `hit_agg_boundary=True` to maintain strict accuracy. The lineage walk continues recursively until it hits a raw source table, seed file, or untraceable node,

returning a precise **BlameLocation**.

Step 3: Warehouse Probes (Evidence Gathering)

Once we have isolated our upstream *BlameLocation*, the agent gathers statistical proof from the warehouse. It runs a suite of defensive, SQL-injection-safe probes on the target column and table. It checks:

Null Rates: The percentage of NULLs in the column, compared to historical z-scores.

Duplicates: Counts of duplicate Primary Keys in the source table.

Orphan Keys: Orphan Foreign Keys that have no corresponding record in parent dimension tables.

Metadata & Fresness: Expected column data types, loading timestamps, table freshness SLA lag, and numeric distributions.

All results are wrapped into a single, token-budgeted, Pydantic-validated **ClassifierEvidence** payload.

Step 4: Classification & The LLM Tiebreaker

The agent runs the evidence through **10 deterministic detectors** (rules). These rules output confidence scores. If a single rule scores very highly (e.g., confidence ≥ 0.85) and outpaces the others, the agent declares a definitive verdict. However, if the rules are uncertain (e.g., top score is low, or top two classes are neck-and-neck), the agent triggers the ****LLM Tiebreaker****.

Using a cost-capped, structured **Claude 3.5 Haiku** model, the agent feeds the evidence JSON to the AI. The model reviews the statistics and outputs a refined Pydantic classification. The API calls are aggressively cached locally to ensure high speed and zero wasted budget.

Step 5: Persist, Narrate, and Heal

The verdict is saved as an immutable **Incident** record in the PostgreSQL or SQLite database using SQLAlchemy and Alembic. Next, the **Narrator** generates a beautiful explanation using Claude 3.5 Sonnet, outlining what broke, why, and a recommended fix. The Slack and GitHub integrations immediately broadcast this narrative.

Finally, the ****Self-Healing Code Generator**** takes the incident verdict and writes a git-ready patch (like adding null-exclusions or correcting a join predicate), pushes a new git branch, and opens a Pull Request automatically.

5. Detailed Taxonomy of the 10 Root-Cause Classes

The agent classifies every data quality incident into one of 10 distinct, structured root-cause families:

- 1. Late-Arriving Data:** Upstream source tables lag far behind the freshness SLA. Real-world case: an ingestion cron job stalled, so new facts are loaded but are missing critical dimension rows.
- 2. Upstream Null Spike:** A massive spike in NULL values occurs in a raw column that should be populated. Example: a source API change starts sending empty strings that map to NULLs.
- 3. Type Coercion / Silent Cast:** Data is implicitly cast in the pipeline, losing precision or causing values to drop out at boundaries. Example: casting a floating-point string directly to an integer.

4. **Duplicate Ingestion:** An extraction job runs twice or lacks de-duplication, loading identical primary keys into the raw table. This violates uniqueness upstream.
5. **Broken Join (Fan-out):** Duplicates on join keys cause a Cartesian product, multiplying the number of rows downstream. This causes severe aggregate distortions.

- 6. Broken Join (Drop-out):** Inner joins or foreign keys refer to parents that do not exist. Downstream records disappear entirely (dropout), or relationships tests fail. Example: a customer ID is referenced but has been soft-deleted from the master table.
- 7. Source-System Schema Change:** An upstream source DDL event modifies a column's data type, name, or drops it entirely, causing downstream dbt compilations to fail.
- 8. Stale Dimension / SCD2 Bug:** A Slowly Changing Dimension (SCD Type 2) table contains incorrect or expired ranges, causing fact records to join against stale dimensions.
- 9. Unit / Encoding Drift:** A numeric column's values shift dramatically (e.g., currency represented in cents instead of dollars, or meters instead of kilometers), exceeding 3-standard-deviation limits.
- 10. Unknown / Multi-Cause:** Default fallback when multiple signals conflict or no specific rules fire. The system gracefully surfaces the raw statistics for human review.

6. Beginner's Quickstart & Usage

Getting started with the DQ Triage Agent is incredibly easy. Let's walk through how to install it, run the test suites, and triage a real dbt failure.

Prerequisites & Installation

Make sure you are in the project folder, then run our convenient installer script. It will automatically create a virtual environment, install all python packages, and configure hooks:

```
make install
source .venv/bin/activate
```

Running the Benchmark Harness

To prove the agent works under various conditions, you can execute our robust fault-injection benchmark. The smoke test runs a subset of trials, while the full suite exercises all 45 injected faults:

```
# Run a quick 18-trial validation (approx. 1 minute)
make eval-smoke

# Run the complete 45-trial suite (approx. 3 minutes)
make eval-full
```

Triageing a Local dbt Test Failure

To triage a real dbt test failure in your own pipeline, simply let dbt execute and fail naturally. This writes the failure details into your dbt target folder. Then, call the agent CLI:

```
dq-triage triage \  
--project /path/to/your/dbt/project \  
--duckdb /path/to/warehouse.duckdb
```

Inspecting Past Incidents

You can inspect previously persisted incidents and show detailed reports directly from the CLI:

```
# List all logged incidents  
dq-triage incidents list  
  
# Show a specific incident report  
dq-triage incidents show inc_114de60254a2
```

IMPORTANT CONFIGURATION

Important Notice: The DQ Triage Agent uses structured schemas and rigid local caching. Before calling LLM features, ensure that the appropriate environment variables (such as `ANTHROPIC_API_KEY`, `SLACK_BOT_TOKEN`, and `GITHUB_TOKEN`) are loaded into your shell session. If no API keys are provided, the system seamlessly falls back to 100% deterministic rules, incurring \$0.00 in execution costs.